

Dynamic Programming Notes

@Wxy2003-xy

Contents

1	Core Dynamic Programming Families	3
1.1	The Knapsack Problem and Its Variants	3
1.2	Longest Increasing Subsequence and Its Variants	4
1.3	Path DP	6
1.4	Interval DP	6
2	Implementation Strategies and State Design	6
2.1	Bottom-Up DP: Iterative DP (Coin Change)	6
2.2	Top-Down DP: Recursion with Memoization (Coin Change)	7
2.3	Choosing Additional State Dimensions	7
3	Classic Dynamic Programming Problem Catalogue	8
3.1	0/1 Knapsack (Maximize Value Within Capacity)	8
3.2	Coin Change (Minimum Coins, Unbounded)	8
3.3	Longest Increasing Subsequence ($O(n^2)$)	8
3.4	Edit Distance (Levenshtein)	9
3.5	Regular Expression Matching (LeetCode 10)	9
3.6	Matrix Chain Multiplication	10
3.7	Longest Common Subsequence	10
3.8	Subset Sum (Feasibility)	10
3.9	Weighted Interval Scheduling	10
3.10	Rod Cutting	11
3.11	Palindrome Partitioning II (Minimum Cuts)	11
3.12	Egg Dropping	11
3.13	Catalan Numbers (Count Valid Parentheses/BSTs)	11
3.14	Word Break (Feasibility with a Dictionary)	11
3.15	Distinct Subsequences (Count $S \rightarrow T$)	12
3.16	Longest Palindromic Subsequence	12
3.17	Tree Minimum Vertex Cover (Rooted)	12
3.18	Traveling Salesman (Bitmask DP)	12
3.19	Grid Minimum Path Sum (Right/Down Moves)	12
3.20	Interleaving String (Feasibility)	13
3.21	Bellman–Ford from a Dynamic Programming Perspective	13
3.22	Floyd–Warshall (All-Pairs Shortest Paths)	14
3.23	Maximum Score from Multiplication Operations (LeetCode 1770)	14
4	High-Dimensional Dynamic Programming	15
4.1	LCS of Three Strings (3D)	16
4.2	0/1 Knapsack with Exactly K Items	16
4.3	2D Knapsack (Weight and Volume)	16
4.4	Count Grid Paths with at Most K Turns	17
4.5	Traveling Salesman (Bitmask DP)	17
4.6	Minimum Cost to Merge Stones (LeetCode 1000)	19
5	Pseudo-Polynomial Dynamic Programming	20
5.1	Overview	20
5.2	Counting (r, ℓ) -Spaced Subsets in $O(rn \log n)$ Time	20

1 Core Dynamic Programming Families

1.1 The Knapsack Problem and Its Variants

The family of knapsack problems is central in algorithm design and dynamic programming. Each variant differs in whether items may be taken once, infinitely many times, or with bounded multiplicities. We discuss the standard recursive formulation, derive each recurrence, and show how the recurrences translate into tabulated DP algorithms.

1.1.1 Classical 0/1 Knapsack

Problem. Given n items, each with weight w_i and value v_i , and knapsack capacity W , find the maximum total value using each item at most once.

Recursive formulation. Let $K(i, w)$ be the maximum value achievable using the first i items within capacity w .
Base cases:

$$K(0, w) = 0, \quad K(i, 0) = 0.$$

If item i is too heavy:

$$K(i, w) = K(i - 1, w).$$

Otherwise:

$$K(i, w) = \max(K(i - 1, w), v_i + K(i - 1, w - w_i)).$$

DP table (bottom-up). Construct a table $dp[i][w] = K(i, w)$ for $i = 0 \dots n$, $w = 0 \dots W$:

$$dp[i][w] = \begin{cases} dp[i - 1][w] & w_i > w, \\ \max(dp[i - 1][w], v_i + dp[i - 1][w - w_i]) & w_i \leq w. \end{cases}$$

Time complexity: $\Theta(nW)$.

1.1.2 Complete (Unbounded) Knapsack

Problem. Each item i may be taken *infinitely many times*. This changes the structure: the choice to include i does not prevent including it again.

Recursive formulation. Base cases remain 0. If $w_i > w$:

$$K(i, w) = K(i - 1, w).$$

Otherwise:

$$K(i, w) = \max(K(i - 1, w), v_i + K(i, w - w_i)).$$

Key difference: the second term uses $K(i, w - w_i)$ instead of $K(i - 1, w - w_i)$ because we may reuse item i .

DP table formulation.

$$dp[i][w] = \max(dp[i - 1][w], v_i + dp[i][w - w_i]).$$

To optimize, we loop w *increasing*:

$$\text{for } i = 1..n : \quad \text{for } w = w_i..W : \quad dp[w] = \max(dp[w], v_i + dp[w - w_i]).$$

This avoids repetition and ensures correctness.

Interpretation. This variant resembles coin change, rod cutting, or unbounded production tasks.

1.1.3 Multiple / Bounded Knapsack

Problem. Each item i may be taken at most c_i times.

Recursive formulation. Naively:

$$K(i, w) = \max_{0 \leq k \leq c_i, kw_i \leq w} (kv_i + K(i - 1, w - kw_i)).$$

This directly follows from enumerating k copies of item i .

DP table formulation. The naive DP:

$$dp[i][w] = \max_{0 \leq k \leq c_i, kw_i \leq w} (dp[i - 1][w - kw_i] + kv_i).$$

This takes $O(nW \cdot \max c_i)$ in general and can be improved by *binary splitting* (a.k.a. binary optimization) of c_i :

$$c_i = (1 + 2 + 4 + \dots + 2^k) + r.$$

This converts bounded knapsack into a series of $O(\log c_i)$ 0/1 knapsack items.

Time complexity. With binary splitting:

$$O\left(W \sum_{i=1}^n \log c_i\right).$$

1.1.4 Relationship Between the Variants

Each recurrence contains the same structural choices but differs in the subproblem invoked when an item is chosen:

Variant	Choice if take item i
0/1	$v_i + K(i-1, w - w_i)$
Unbounded	$v_i + K(i, w - w_i)$
Bounded	$\max_{k \leq c_i} (kv_i + K(i-1, w - kw_i))$

Conceptually:

- 0/1 Knapsack disallows reuse $\Rightarrow$ previous row only.
- Complete Knapsack allows reuse $\Rightarrow$ current row reused.
- Bounded Knapsack is between them, requiring enumeration or binary expansion.

1.1.5 Summary

Knapsack DP always follows the same overarching pattern:

$$\text{value}(w) = \max\{\text{skip item, take item (once/many/limited)}\}.$$

What differs is the *subproblem* that taking an item leads to:

$$K(i-1, w - w_i) \text{ (0/1), } K(i, w - w_i) \text{ (unbounded), } K(i-1, w - kw_i) \text{ (bounded).}$$

The transition between the recurrence and the tabulated DP is systematic: rows correspond to items, columns correspond to capacities, and the change of index in the “take” case reflects the allowed multiplicities.

1.2 Longest Increasing Subsequence and Its Variants

The Longest Increasing Subsequence (LIS) problem is a fundamental example of dynamic programming, binary search optimization, and multi-dimensional ordering. This subsection summarizes its DP formulation, the $O(n \log n)$ algorithm, and common extensions such as 2D LIS on point sets and on 2D grids.

1.2.1 Longest Increasing Subsequence (1D)

Problem. Given an array $A[1..n]$, find the longest subsequence

$$i_1 < i_2 < \dots < i_k \quad \text{such that} \quad A[i_1] < A[i_2] < \dots < A[i_k].$$

DP recurrence (classical $O(n^2)$). Let $dp[i]$ be the length of the LIS ending at index i .

$$dp[i] = 1 + \max_{j < i, A[j] < A[i]} dp[j], \quad dp[i] = 1 \text{ if no such } j.$$

The final answer is $\max_i dp[i]$.

Binary-search optimization ($O(n \log n)$). Maintain an array T where $T[\ell]$ stores the minimum possible ending value of an increasing subsequence of length ℓ .

Algorithm:

1. Initialize an empty array T .
2. For each $A[i]$: use binary search to find the smallest ℓ such that $T[\ell] \geq A[i]$.
3. Set $T[\ell] = A[i]$.

The length of T equals the LIS length. This method exploits the monotonic structure of the DP.

1.2.2 Longest Contiguous Increasing Subarray with Switching Between Price Sources

We are given three arrays `price1`, `price2`, and `price3` of equal length n , representing the stock price reported by three different market makers on each day. On every day i , we may freely choose one of the three prices to form a *combined* array `price`. Our goal is to determine the length of the longest contiguous non-empty subarray of `price` whose values are strictly increasing.

DP formulation. Let the three sources be indexed by $k \in \{1, 2, 3\}$ and the days by $i \in \{1, \dots, n\}$. Define

$dp[k][i]$ = length of the longest valid strictly increasing contiguous segment ending at day i if we choose price source k on day i .

Base case. A single day alone forms a strictly increasing subarray of length 1:

$$\text{dp}[k][1] = 1 \quad \forall k \in \{1, 2, 3\}.$$

Recurrence relation. For each day $i > 1$, if we choose price source k on day i , then we may extend a segment ending at day $i - 1$ only if the chosen price is strictly larger than the previous day's value:

$$\text{dp}[k][i] = 1 + \max_{\substack{j \in \{1, 2, 3\} \\ \text{price}_j[i-1] < \text{price}_k[i]}} \text{dp}[j][i-1].$$

If no j satisfies $\text{price}_j[i-1] < \text{price}_k[i]$, then the segment must start anew at day i :

$$\text{dp}[k][i] = 1.$$

Final answer. The length of the longest valid segment is

$$\max_{i=1..n} \max_{k=1..3} \text{dp}[k][i].$$

Complexity. The DP table has only $3n$ states, and each state checks only the three possible previous choices, giving a linear running time:

$$O(n).$$

Space usage can be $O(3n)$ or reduced to $O(1)$ by storing only values for day i and day $i - 1$.

Summary. This problem is solved by a small constant-state dynamic program. At each day, the decision depends only on the strictly increasing condition against the previous day's three possible prices. Contiguousness is enforced naturally because each $\text{dp}[k][i]$ extends only from day $i - 1$, and switching between sources is free.

1.2.3 LIS in 2D: Longest Increasing Sequence in the Plane

A common generalization is LIS over points (x_i, y_i) . We seek the largest subset of points ordered so that

$$x_{i_1} < x_{i_2} < \dots < x_{i_k} \quad \text{and} \quad y_{i_1} < y_{i_2} < \dots < y_{i_k}.$$

Sort + LIS trick. Sort points by:

$$\text{increasing } x, \quad \text{and for equal } x, \text{ decreasing } y.$$

This prevents selecting two points with equal x or forming invalid ties.

After sorting, the problem reduces to LIS on the y -coordinates:

$$\text{LIS on } (y_1, y_2, \dots, y_n).$$

Time:

$$O(n \log n) \quad \text{via the binary search LIS algorithm.}$$

Applications.

- Russian-doll envelopes (LeetCode 354),
- scheduling where both start- and end-times must increase,
- multi-criteria filtering and skyline queries,
- geometry and dominance problems.

1.2.4 LIS in a 2D Grid (Monotone Paths)

Given an $m \times n$ grid A , a “2D LIS path” requires increasing values along a path moving only rightward or downward:

$$A[i_1][j_1] < A[i_2][j_2] < \dots, \quad i_{t+1} \geq i_t, \quad j_{t+1} \geq j_t.$$

DP formulation. Let $\text{dp}[i][j]$ be the length of the longest increasing path ending at (i, j) .

$$\text{dp}[i][j] = 1 + \max \left(\text{dp}[i-1][j] \text{ if } A[i-1][j] < A[i][j], \quad \text{dp}[i][j-1] \text{ if } A[i][j-1] < A[i][j] \right).$$

Boundary:

$$\text{dp}[i][j] = 1 \quad \text{if neither neighbor is smaller.}$$

Time complexity:

$$O(mn).$$

Remarks. This is not the same as LIS on the array formed by flattening the matrix, because 2D movement constraints restrict the transitions.

1.2.5 Longest Chain in Higher Dimensions

For d -dimensional points $p_i = (a_{i,1}, a_{i,2}, \dots, a_{i,d})$, we want the longest chain satisfying

$$p_{i_1} < p_{i_2} < \dots < p_{i_k} \quad \text{in all coordinates.}$$

Typical solution:

- Sort lexicographically by $(a_1, a_2, \dots, a_{d-1})$,
- Apply LIS to the last coordinate, possibly with transformations when ties occur.

This generalizes the 2D LIS sorting trick.

1.2.6 Summary of Patterns

- 1D LIS: $dp[i] = 1 + \max_{j < i, A[j] < A[i]} dp[j]$ or $O(n \log n)$ via patience sorting.
- 2D LIS: sort by one coordinate, apply 1D LIS to the other.
- Grid LIS: DP with directional constraints; no binary-search reduction.
- Higher-dimensional LIS: sort by all but one coordinate, LIS on last.

Across all variants, the structural pattern is:

Sort or order the states so that the remaining dimension becomes 1D LIS.

This unifies many “increasing sequence” problems.

1.3 Path DP

For Floyd–Warshall, let $T_k(i, j)$ denote the shortest path from i to j whose intermediate vertices are drawn from $\{1, \dots, k\}$. Then:

$$T_0(i, j) = \begin{cases} 0 & i = j, \\ w(i, j) & (i, j) \text{ is an edge,} \\ \infty & \text{otherwise,} \end{cases}$$

$$T_k(i, j) = \min(T_{k-1}(i, j), T_{k-1}(i, k) + T_{k-1}(k, j)).$$

1.4 Interval DP

Example: convex polygon triangulation

$$T[x, y] = 0 \quad \text{if } y \leq x + 1,$$

$$T[x, y] = \min_{x < k < y} (T[x, k] + W(x, k, y) + T[k, y]).$$

2 Implementation Strategies and State Design

2.1 Bottom-Up DP: Iterative DP (Coin Change)

given the recurrence: $T(n) = \min_{c \in \text{coins}} (T(n - c) + 1)$, we know the state we need to remember is the min number of coins to get $n - c$. Only 1 param $\rightarrow$ 1D DP

```
int coinChangeMin(vector<int>& coins, int amount) {
    const int INF = 1e9;           // sentinel for impossible
    vector<int> dp(amount + 1, INF);
    dp[0] = 0;                     // base: 0 coins to make 0

    for (int a = 1; a <= amount; ++a) {
        for (int c : coins) {
            if (c <= a && dp[a - c] != INF) {
                dp[a] = min(dp[a], dp[a - c] + 1);
            }
        }
    }
    return (dp[amount] == INF ? -1 : dp[amount]);
}
```

2.2 Top-Down DP: Recursion with Memoization (Coin Change)

```
int helper(int amount, vector<int>& coins, vector<int>& memo) {
    if (amount == 0) return 0;          // base: 0 coins to make 0
    if (amount < 0) return 1e9;         // sentinel: impossible
    if (memo[amount] != -1) return memo[amount];

    int best = 1e9;
    for (int c : coins) {
        best = min(best, 1 + helper(amount - c, coins, memo));
    }
    return memo[amount] = best;
}

int coinChangeMin(vector<int>& coins, int amount) {
    vector<int> memo(amount + 1, -1);
    int res = helper(amount, coins, memo);
    return (res >= 1e9 ? -1 : res); // -1 means impossible
}
```

Notice that in coin change, we only need to maintain a 1D memo table for top-down dp and 1D dp table, as in the naive recursive solution we found out there is only 1 state needs to be remembered, therefore 1 dimensional auxiliary ds is needed. In comparison, for knapsack, where 2 states need to be remembered, remaining weight and current value sum, we need a 2D table to implement dp.

```
int helper(int k, const vector<vector<int>>& c, int cur) {
    if (k < 0) {
        return -1;
    }
    if (k >= 0 && cur == 0) {
        return 0;
    }
    int best = 0;
    return max(helper(k - c[cur][0], c, cur - 1) + c[cur][1],
               helper(k, c, cur - 1));
}

int knapsack(int k, const vector<vector<int>>& c) {
    return helper(k, c, c.size()-1);
}

int knapsack_bu(int k, const vector<vector<int>>& c) {
    vector<vector<int>> dp(c.size() + 1, vector<int>(k+1, -1));
    for (int i = 0; i < k; i++) {
        dp[0][i] = 0;
    }
    for (int i = 0; i < c.size() + 1; i++) {
        dp[i][0] = 0;
    }
    for (int i = 1; i < c.size()+1; i++) {
        for (int j = 1; j < k+1; j++) {
            if (j >= c[i-1][0]) {
                dp[i][j] = max(dp[i-1][j], dp[i-1][j-c[i-1][0]] + c[i-1][1]);
            } else {
                dp[i][j] = dp[i-1][j];
            }
        }
    }
    return dp[c.size()][k-1];
}
```

From another view, to uniquely identify an instance of coin change problem (on the same set of coins available), 1 key is needed (total value); while to uniquely identify an instance of knapsack (on the same set of items), we need 2 keys. This alludes that we can choose the dimension of our dp table based on the number of states need to be maintained.

2.3 Choosing Additional State Dimensions

In max profit problem,

```
long long maximumProfit(vector<int>& p, int k) {
    int n = p.size();
    if (n < 2 || k == 0) return 0;
    const long long NEG_INF = -(1LL << 60);

    // dp[day][transactions][state]
    // state: 0 = NOT_HOLDING, 1 = HOLDING, 2 = HOLDING_SHORT
    vector<vector<vector<long long>>> dp(
        n, vector<vector<long long>>(k+1, vector<long long>(3, NEG_INF)))
```

```

);

// pad base case: max profit no transaction yet
dp[0][0][0] = 0; // NOT_HOLDING
dp[0][0][1] = -(long long)p[0]; // HOLDING
dp[0][0][2] = (long long)p[0]; // HOLDING_SHORT

for (int j = 1; j < n; j++) { // day
    for (int i = 0; i <= k; i++) { // transactions
        // if continue to enter NOT_HOLDING
        dp[j][i][0] = max(dp[j][i][0], dp[j-1][i][0]);
        // if choose to enter HOLDING
        dp[j][i][1] = max(dp[j-1][i][1], dp[j-1][i][0] - (long long)p[j]);
        // if choose to enter HOLDING_SHORT
        dp[j][i][2] = max(dp[j-1][i][2], dp[j-1][i][0] + (long long)p[j]);
        if (i >= 1) { // selling only possible after first day
            // if choose to enter NOT_HOLDING
            dp[j][i][0] = max(dp[j][i][0], dp[j-1][i-1][1] + (long long)p[j]);
            dp[j][i][0] = max(dp[j][i][0], dp[j-1][i-1][2] - (long long)p[j]);
        }
    }
}

long long ans = 0;
for (int i = 1; i <= k; i++) {
    ans = max(ans, dp[n-1][i][0]); // need to end with NOT_HOLDING state. (Invariant 5)
}
return ans;
}

```

There are 2 obvious states, day and number of transactions. But there are additional constraint in the question, which is the current action state, whether we are holding a stock. This state affects decision making as it is 1. independent from other states, ie any day we may be in any action state; 2. the transition among this action states is one of the constraints posed by the problem.

Here's a quick nudge first: for each problem, name a **state** that captures exactly "what's left to decide," ensure **overlap** (reuse subresults), and pick a **loop order** that enforces the selection rules (0/1 vs unbounded, interval growth, masks, post-order on trees).

3 Classic Dynamic Programming Problem Catalogue

3.1 0/1 Knapsack (Maximize Value Within Capacity)

Note: DP solution runs in pseudo-poly time, the problem is NPC

Description. Choose a subset of items (each at most once) to maximize total value without exceeding capacity.

Recurrence/base. Let $dp[i][c]$ be max value using first i items within capacity c . $dp[i][c] = \max(dp[i-1][c], dp[i-1][c-w_i] + v_i)$ if $w_i \leq c$, else $dp[i][c] = dp[i-1][c]$.

Base: $dp[0][c] = 0$, $dp[i][0] = 0$.

```

for i=1..n:
    for c=0..K:
        dp[i][c] = dp[i-1][c]
        if w[i] <= c:
            dp[i][c] = max(dp[i][c], dp[i-1][c-w[i]] + v[i])
answer = dp[n][K]

```

3.2 Coin Change (Minimum Coins, Unbounded)

Description. Using unlimited copies of given coin denominations, make amount K with fewest coins.

Recurrence/base. $dp[a] = \min_{d \leq a} (1 + dp[a-d])$.

Base: $dp[0] = 0$; others $+\infty$.

```

dp[0]=0; for a=1..K: dp[a]=INF
for a=1..K:
    for d in coins:
        if d <= a and dp[a-d] < INF:
            dp[a] = min(dp[a], dp[a-d] + 1)
answer = (dp[K]<INF ? dp[K] : -1)

```

3.3 Longest Increasing Subsequence ($O(n^2)$)

Description. Longest subsequence with strictly increasing values.

Recurrence/base. $dp[i] = 1 + \max_{j < i, a_j < a_i} dp[j]$;

base $dp[i] = 1$.


```

for i=1..n:
    dp[i]=1
    for j=1..i-1:
        if a[j] < a[i]: dp[i]=max(dp[i], dp[j]+1)
answer = max_i dp[i]

```

3.4 Edit Distance (Levenshtein)

Description. Minimum insert/delete/replace operations to convert string s to t .

Recurrence/base. $dp[i][j] = \min\{dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + [s_i \neq t_j]\}$.

Base: $dp[0][j] = j, dp[i][0] = i$.

```

for i=0..n: dp[i][0]=i
for j=0..m: dp[0][j]=j
for i=1..n:
    for j=1..m:
        cost = (s[i]==t[j]?0:1)
        dp[i][j]=min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost)
answer = dp[n][m]

```

3.5 Regular Expression Matching (LeetCode 10)

Problem. Given a text s and a pattern p , determine whether the pattern matches the *entire* text. A dot $.$ matches any single character, while $*$ matches zero or more copies of the immediately preceding element.

State. Let

$$dp[i][j] = \begin{cases} \text{true} & \text{if } s[0..i-1] \text{ matches } p[0..j-1], \\ \text{false} & \text{otherwise.} \end{cases}$$

Define

$$\text{match}(i, j) \equiv i > 0 \wedge (p[j-1] = . \vee p[j-1] = s[i-1]).$$

Base Cases. The two empty prefixes match:

$$dp[0][0] = \text{true}.$$

A nonempty text cannot match an empty pattern, so $dp[i][0] = \text{false}$ for $i > 0$. An empty text can match patterns such as $a^*b^*c^*$:

$$dp[0][j] = dp[0][j-2] \quad \text{when } p[j-1] = *.$$

Recurrence. If the current pattern character is not $*$, it must consume exactly one text character:

$$dp[i][j] = \text{match}(i, j) \wedge dp[i-1][j-1].$$

If $p[j-1] = *$, there are two possibilities:

$$dp[i][j] = dp[i][j-2] \vee (\text{match}(i, j-1) \wedge dp[i-1][j]).$$

The first term uses zero copies of the starred element. The second consumes one matching text character and keeps the same pattern prefix, allowing more copies to be consumed.

Bottom-Up DP.

```

bool isMatch(const string& s, const string& p) {
    int n = s.size(), m = p.size();
    vector<vector<char>> dp(n + 1, vector<char>(m + 1, false));
    dp[0][0] = true;

    for (int j = 2; j <= m; ++j)
        if (p[j-1] == '*')
            dp[0][j] = dp[0][j-2];

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (p[j-1] != '*') {
                bool same = p[j-1] == '.' || p[j-1] == s[i-1];
                dp[i][j] = same && dp[i-1][j-1];
            } else if (j >= 2) {
                dp[i][j] = dp[i][j-2]; // zero copies
            }
        }
    }
}

```

```

        bool same = p[j - 2] == '.' || p[j - 2] == s[i - 1];
        if (same)
            dp[i][j] = dp[i][j] || dp[i - 1][j];
    }
}
return dp[n][m];
}

```

Correctness. Every non-starred pattern element has exactly one legal action: match and consume one text character. A starred element either contributes zero copies or contributes at least one copy; these cases are disjoint and cover every valid match. Since each transition refers to shorter prefixes, induction on $i + j$ proves that every table entry has the stated meaning.

Complexity. There are $(n + 1)(m + 1)$ states and $O(1)$ work per state:

$$O(nm) \text{ time, } \quad O(nm) \text{ space.}$$

The space can be reduced to $O(m)$ by retaining only the current and previous rows.

3.6 Matrix Chain Multiplication

Description. Parenthesize a chain of matrices to minimize scalar multiplications.

Recurrence/base. $dp[i][j] = \min_{i \leq k < j} \{dp[i][k] + dp[k+1][j] + p_{i-1}p_kp_j\}$.

Base: $dp[i][i] = 0$.

```

for i=1..n: dp[i][i]=0
for len=2..n:
    for i=1..n-len+1:
        j=i+len-1; dp[i][j]=INF
        for k=i..j-1:
            dp[i][j]=min(dp[i][j], dp[i][k]+dp[k+1][j]+p[i-1]*p[k]*p[j])
answer = dp[1][n]

```

3.7 Longest Common Subsequence

Description. Longest sequence that appears as a subsequence in both strings.

Recurrence/base. If $s_i = t_j$: $dp[i][j] = dp[i-1][j-1] + 1$, else $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

Base: $dp[0][*] = dp[*][0] = 0$.

```

for i=0..n: dp[i][0]=0
for j=0..m: dp[0][j]=0
for i=1..n:
    for j=1..m:
        if s[i]==t[j]: dp[i][j]=dp[i-1][j-1]+1
        else: dp[i][j]=max(dp[i-1][j], dp[i][j-1])
answer = dp[n][m]

```

3.8 Subset Sum (Feasibility)

Note: DP solution runs in pseudo-poly time, the problem is NPC

Description. Decide if some subset sums exactly to K .

Recurrence/base. $F[i][c] = F[i-1][c] \vee (w_i \leq c \wedge F[i-1][c-w_i])$.

Base: $F[0][0] = \text{true}$, $F[0][c > 0] = \text{false}$.

```

F[0][0]=true; for c=1..K: F[0][c]=false
for i=1..n:
    for c=0..K:
        F[i][c]=F[i-1][c]
        if w[i]<=c and F[i-1][c-w[i]]: F[i][c]=true
answer = F[n][K]

```

3.9 Weighted Interval Scheduling

Description. Pick non-overlapping weighted intervals to maximize total weight.

Recurrence/base. Sort by finish; let $p(i)$ be last non-overlapping.

$OPT[i] = \max(OPT[i-1], v_i + OPT[p(i)])$. Base $OPT[0] = 0$.

```

OPT[0]=0
for i=1..n:
    OPT[i]=max( OPT[i-1], v[i] + OPT[p(i)] )
answer = OPT[n]

```

3.10 Rod Cutting

Description. Cut a rod into lengths to maximize sale price.

Recurrence/base. $dp[L] = \max_{1 \leq j \leq L} (price[j] + dp[L - j])$.

Base: $dp[0] = 0$.

```

dp[0]=0
for L=1..N:
    dp[L]=0
    for j=1..L:
        dp[L]=max(dp[L], price[j]+dp[L-j])
answer = dp[N]

```

3.11 Palindrome Partitioning II (Minimum Cuts)

Description. Split a string into palindromic substrings with minimum cuts.

Recurrence/base. Precompute $pal[i][j]$.

$cuts[i] = \min_{0 \leq j \leq i, pal[j][i]} ((j == 0 ? 0 : cuts[j - 1] + 1))$.

Base: handled via $j == 0$.

```

build pal[n][n] // O(n^2)
for i=0..n-1:
    cuts[i]=INF
    for j=0..i:
        if pal[j][i]:
            cuts[i]=min(cuts[i], (j==0?0:cuts[j-1]+1))
answer = cuts[n-1]

```

3.12 Egg Dropping

Description. Minimize worst-case trials to find critical floor with given eggs.

Recurrence/base. $dp[e][f] = 1 + \min_{1 \leq x \leq f} \max(dp[e - 1][x - 1], dp[e][f - x])$.

Base: $dp[1][f] = f$, $dp[e][0] = 0$.

```

for f=0..F: dp[1][f]=f
for e=2..E: dp[e][0]=0
for e=2..E:
    for f=1..F:
        dp[e][f]=INF
        for x=1..f:
            dp[e][f]=min(dp[e][f], 1+max(dp[e-1][x-1], dp[e][f-x]))
answer = dp[E][F]

```

3.13 Catalan Numbers (Count Valid Parentheses/BSTs)

Description. Count structures like valid parentheses strings or BST shapes.

Recurrence/base. $C_0 = 1$, $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$.

```

C[0]=1
for n=1..N:
    C[n]=0
    for i=0..n-1:
        C[n]+=C[i]*C[n-1-i]
answer = C[N]

```

3.14 Word Break (Feasibility with a Dictionary)

Description. Check if a string can be segmented into dictionary words.

Recurrence/base. $dp[i] = \bigvee_{0 \leq j < i} (dp[j] \wedge s[j..i - 1] \in D)$.

Base: $dp[0] = \text{true}$.

```

dp[0]=true; for i=1..n: dp[i]=false
for i=1..n:
    for j=0..i-1:
        if dp[j] and (s[j..i-1] in D): dp[i]=true
answer = dp[n]

```

3.15 Distinct Subsequences (Count $S \rightarrow T$)

Description. Number of ways string T appears as a subsequence of S .

Recurrence/base. $dp[i][j] = dp[i-1][j] + [s_i = t_j] \cdot dp[i-1][j-1]$.

Base: $dp[i][0] = 1$, $dp[0][j > 0] = 0$.

```
for i=0..n: dp[i][0]=1
for j=1..m: dp[0][j]=0
for i=1..n:
    for j=1..m:
        dp[i][j]=dp[i-1][j]
        if s[i]==t[j]: dp[i][j]+=dp[i-1][j-1]
answer = dp[n][m]
```

3.16 Longest Palindromic Subsequence

Description. Longest subsequence that is a palindrome.

Recurrence/base. If $s_i = s_j$: $dp[i][j] = 2 + dp[i+1][j-1]$ else $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$.

Base: $dp[i][i] = 1$; $dp[i][i-1] = 0$.

```
for i=0..n-1: dp[i][i]=1
for len=2..n:
    for i=0..n-len:
        j=i+len-1
        if s[i]==s[j]:
            dp[i][j]= (len==2?2:2+dp[i+1][j-1])
        else:
            dp[i][j]=max(dp[i+1][j], dp[i][j-1])
answer = dp[0][n-1]
```

3.17 Tree Minimum Vertex Cover (Rooted)

Description. Smallest set of vertices covering all edges in a tree.

Recurrence/base. $dp[u][1] = 1 + \sum_v \min(dp[v][0], dp[v][1])$,

$dp[u][0] = \sum_v dp[v][1]$.

Base: leaf $dp[leaf][1] = 1$, $dp[leaf][0] = 0$.

```
postorder over tree:
    if leaf: dp[u][0]=0; dp[u][1]=1
    else:
        take = 1 + sum_v min(dp[v][0], dp[v][1])
        skip = sum_v dp[v][1]
        dp[u][0]=skip; dp[u][1]=take
answer = min(dp[root][0], dp[root][1])
```

3.18 Traveling Salesman (Bitmask DP)

Description. Shortest Hamiltonian tour visiting all nodes once and returning to start.

Recurrence/base. $dp[mask][i] = \min_{j \in mask \setminus \{i\}} dp[mask \setminus \{i\}][j] + dist[j][i]$.

Base: $dp[1 \ll s][s] = 0$.

```
for mask=0..(1<<n)-1:
    for i in nodes:
        dp[mask][i]=INF
dp[1<<s][s]=0
for mask from 1..(1<<n)-1:
    for i in mask:
        for j in mask, j!=i:
            dp[mask][i]=min(dp[mask][i], dp[mask^(1<<i)][j]+dist[j][i])
answer = min_i dp[(1<<n)-1][i] + dist[i][s]
```

3.19 Grid Minimum Path Sum (Right/Down Moves)

Description. Minimum sum path from top-left to bottom-right moving only right or down.

Recurrence/base. $dp[i][j] = w_{i,j} + \min(dp[i-1][j], dp[i][j-1])$.

Base: first row/col cumulative.

```
dp[0][0]=w[0][0]
for i=1..R-1: dp[i][0]=dp[i-1][0]+w[i][0]
for j=1..C-1: dp[0][j]=dp[0][j-1]+w[0][j]
for i=1..R-1:
```

```

for j=1..C-1:
    dp[i][j]=w[i][j]+min(dp[i-1][j], dp[i][j-1])
answer = dp[R-1][C-1]

```

3.20 Interleaving String (Feasibility)

Description. Decide if s_3 can be formed by interleaving s_1 and s_2 in order.

Recurrence/base. $dp[i][j] = (dp[i-1][j] \wedge s1_i = s3_{i+j}) \vee (dp[i][j-1] \wedge s2_j = s3_{i+j})$.

Base: $dp[0][0] = \text{true}$; first row/col by equality chains.

```

dp[0][0]=true
for i=1..n: dp[i][0]= dp[i-1][0] and s1[i]==s3[i]
for j=1..m: dp[0][j]= dp[0][j-1] and s2[j]==s3[j]
for i=1..n:
    for j=1..m:
        dp[i][j]= (dp[i-1][j] and s1[i]==s3[i+j]) or
                  (dp[i][j-1] and s2[j]==s3[i+j])
answer = dp[n][m]

```

3.21 Bellman–Ford from a Dynamic Programming Perspective

Problem. Given a directed graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$ and a source $s \in V$, compute the shortest-path distance from s to all vertices, even when some edge weights are negative (but no negative cycles reachable from s).

DP Formulation. Define a DP table $d[k][v]$ as:

$$d[k][v] = \text{shortest-path distance from } s \text{ to } v \text{ using at most } k \text{ edges.}$$

We initialize:

$$d[0][s] = 0, \quad d[0][v] = +\infty \text{ for } v \neq s.$$

Recurrence. For $k \geq 1$:

$$d[k][v] = \min \left(d[k-1][v], \min_{(u,v) \in E} (d[k-1][u] + w(u,v)) \right).$$

This captures the idea:

- Either the shortest k -edge path to v uses $\leq k-1$ edges, or
- It ends with a single edge (u, v) and a $(k-1)$ -edge path to u .

Key Observation. Any simple shortest path in a graph with $|V| = n$ has at most $n-1$ edges. Thus:

$$d[n-1][v] = \text{true shortest-path distance from } s \text{ to } v.$$

Bellman–Ford Algorithm. Rather than store a full $n \times n$ DP table, we keep only the previous row:

$$d[v] \leftarrow d[v] \text{ at iteration } k-1, \quad d'[v] \leftarrow d[v] \text{ at iteration } k.$$

Each iteration performs:

$$\forall (u, v) \in E : \quad d[v] \leftarrow \min (d[v], d[u] + w(u, v)).$$

We repeat this for $n-1$ iterations. By the DP lemma above, after iteration $n-1$ the distances are optimal.

Detecting Negative Cycles. If a further iteration (the n th) still changes some $d[v]$, then:

$$d[n][v] < d[n-1][v],$$

which means that there exists a path with more than $n-1$ improving relaxations—impossible unless a negative-weight cycle is reachable from s .

DP Interpretation. The classical Bellman–Ford relaxation loop

$$\text{repeat } n-1 \text{ times: for every edge } (u, v), \text{ relax } d[v]$$

is exactly computing the DP layers $d[1], d[2], \dots, d[n-1]$: each layer incorporates paths using at most one more edge.

Thus Bellman–Ford is a straightforward instance of dynamic programming on path length: it monotonically improves the DP table until no better solution exists.

Time Complexity. The DP recurrence is evaluated $n - 1$ times over the edge set, giving:

$$T(n, m) = O(nm),$$

which is optimal for this DP formulation.

Conclusion. Bellman–Ford is dynamic programming over path length: it computes shortest paths by repeatedly applying the DP recurrence for paths of increasing edge count, guaranteeing correctness even with negative weights.

3.22 Floyd–Warshall (All-Pairs Shortest Paths)

Description. Computes shortest-path distances between *all* pairs of vertices in a weighted graph (directed or undirected), allowing negative edges but *no* negative cycles reachable between pairs.

DP perspective. Let $dp[k][i][j]$ = shortest distance from i to j using only intermediate vertices from the set $\{1, \dots, k\}$. Transition considers whether we use vertex k as an intermediate node:

$$dp[k][i][j] = \min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j]).$$

Final answer is $dp[V][i][j]$ for all pairs (i, j) .

Space optimization. We only need the previous layer $k - 1$; thus we collapse into a 2D table:

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j]).$$

Base initialization.

$$dist[i][i] = 0, \quad dist[i][j] = w(i, j) \text{ if edge exists, else } \infty.$$

```
for i = 1..n:
  for j = 1..n:
    if i == j: dist[i][j] = 0
    else if edge(i,j): dist[i][j] = w(i,j)
    else: dist[i][j] = INF

for k = 1..n:          # allow intermediate vertices up to k
  for i = 1..n:
    for j = 1..n:
      if dist[i][k] + dist[k][j] < dist[i][j]:
        dist[i][j] = dist[i][k] + dist[k][j]

# dist[i][j] now contains APSP distances
```

Answer. The full APSP matrix $dist[i][j]$ for all vertex pairs.

Time/Space.

$$O(|V|^3) \text{ time,} \quad O(|V|^2) \text{ space.}$$

3.23 Maximum Score from Multiplication Operations (LeetCode 1770)

Problem. We are given:

- an integer array `nums` of length n ,
- an integer array `multipliers` of length m , with $m \leq n$.

We perform exactly m operations. On the i -th operation ($0 \leq i < m$):

- we choose either the leftmost or rightmost element from the current `nums`,
- multiply it by `multipliers[i]`,
- add it to our total score, and remove that chosen element from `nums`.

We want to maximize the total score after m operations.

State Definition. A key observation is that after i operations, we have removed exactly i elements in total, split between the left end and the right end. The remaining middle segment is uniquely determined by how many we have removed from the left.

Let `nums` be 0-indexed of length n , `multipliers` 0-indexed of length m . Define a DP state:

$dp[i][\ell]$ = maximum score after performing i operations and having taken ℓ numbers from the left.

Then:

- We have taken $i - \ell$ numbers from the right.
- The next multiplier we will use is `multipliers[i]`.
- The leftmost available index is ℓ .
- The rightmost available index is

$$r = (n - 1) - (i - \ell).$$

Base Case. Before performing any operations:

$$\text{dp}[0][0] = 0,$$

and all other $\text{dp}[0][\ell]$ for $\ell > 0$ are invalid (can be treated as $-\infty$).

Recurrence. At step i ($0 \leq i < m$), suppose we are computing $\text{dp}[i+1][\ell']$. There are two ways to reach it:

1. **Pick from the left.** If at step i we had taken $\ell' - 1$ elements from the left, then we now take the next left element $\text{nums}[\ell' - 1]$ with multiplier $\text{multipliers}[i]$:

$$\text{dp}[i+1][\ell'] \supset \text{dp}[i][\ell' - 1] + \text{multipliers}[i] \cdot \text{nums}[\ell' - 1].$$

2. **Pick from the right.** If at step i we had taken ℓ' elements from the left, then we now take a right element. At that moment, we have taken $i - \ell'$ from the right so far, so the rightmost available index is:

$$r = (n - 1) - (i - \ell').$$

We gain:

$$\text{dp}[i+1][\ell'] \supset \text{dp}[i][\ell'] + \text{multipliers}[i] \cdot \text{nums}[r].$$

Combining both choices, for valid indices:

$$\boxed{\text{dp}[i+1][\ell] = \max \left(\text{dp}[i][\ell - 1] + \text{multipliers}[i] \cdot \text{nums}[\ell - 1], \text{dp}[i][\ell] + \text{multipliers}[i] \cdot \text{nums}[(n - 1) - (i - \ell)] \right)}$$

We fill this DP table for $i = 0, \dots, m - 1$ and $\ell = 0, \dots, i + 1$. The answer is:

$$\boxed{\max_{0 \leq \ell \leq m} \text{dp}[m][\ell]}$$

since after m operations we may have taken any feasible number ℓ from the left.

Why the Recurrence is Correct. After i operations, the configuration of remaining nums is completely determined by two numbers:

- how many picks we have made (i),
- how many of those picks were from the left (ℓ).

Thus, the remaining available segment is from index ℓ to index $r = (n - 1) - (i - \ell)$. The future decisions and score depend only on this segment and which multiplier index i we are at. This justifies that $\text{dp}[i][\ell]$ captures all necessary information (optimal substructure).

Each transition corresponds exactly to the only two available choices (pick left or pick right), and we take the maximum over them, so by standard DP cut-and-paste argument, this recurrence computes the optimal value.

Time and Space Complexity. The DP table has dimensions:

$$i = 0, \dots, m \quad \text{and} \quad \ell = 0, \dots, i,$$

so the number of states is:

$$\sum_{i=0}^m (i + 1) = O(m^2).$$

Each state is computed in $O(1)$ time from two previous states, giving total time:

$$\boxed{O(m^2)}.$$

Space usage is $O(m^2)$ if we store the full 2D table. Since $\text{dp}[i+1][\ell]$ depends only on row i , we can optimize to $O(m)$ space by rolling arrays.

In summary, the DP algorithm runs in $O(m^2)$ time and $O(m)$ - $O(m^2)$ space and yields the maximum possible score.

4 High-Dimensional Dynamic Programming

Hint. Go up a dimension only when a real constraint requires it (extra budget, exact count, direction/turns, multiple strings, or a set-state like a bitmask). Watch base layers and sentinel values—most bugs are off-by-one or mixing "previous layer" with "current layer."

4.1 LCS of Three Strings (3D)

Description. Longest common subsequence among S, T, U .

State. $dp[i][j][k]$ = LCS length of $S[1..i], T[1..j], U[1..k]$.

Recurrence.

$$dp[i][j][k] = \begin{cases} dp[i-1][j-1][k-1] + 1 & \text{if } S_i = T_j = U_k \\ \max\{dp[i-1][j][k], dp[i][j-1][k], dp[i][j][k-1]\} & \text{otherwise} \end{cases}$$

Base. Any index 0 gives 0.

```
int lcs3(const string& A, const string& B, const string& C){
    int n=A.size(), m=B.size(), l=C.size();
    vector dp(n+1, vector(m+1, vector<int>(l+1, 0)));
    for(int i=1; i<=n; i++)
        for(int j=1; j<=m; j++)
            for(int k=1; k<=l; k++){
                if(A[i-1]==B[j-1] && B[j-1]==C[k-1])
                    dp[i][j][k]=dp[i-1][j-1][k-1]+1;
                else
                    dp[i][j][k]=max({dp[i-1][j][k],
                                     dp[i][j-1][k],
                                     dp[i][j][k-1]});
            }
    return dp[n][m][l];
}
```

4.2 0/1 Knapsack with Exactly K Items

Description. Choose exactly K_{items} items, total weight $\leq K$, maximize value.

State. $dp[i][c][k]$ = best value using first i items, capacity c , picking exactly k items. **Recurrence.**

$$dp[i][c][k] = \max \left(dp[i-1][c][k], dp[i-1][c-w_i][k-1] + v_i \right)$$

Base. $dp[0][0][0] = 0$; others $-\infty$.

(2-layer roll on i).

```
int knap_exact_k(const vector<int>& W,
                const vector<int>& V, int Kcap, int Kitems){
    const int n=W.size(), NEG=-1e9;
    vector prev(Kcap+1, vector<int>(Kitems+1, NEG)), cur=prev;
    prev[0][0]=0;
    for(int i=1; i<=n; i++){
        cur=prev;
        for(int c=0; c<=Kcap; c++){
            for(int k=0; k<=Kitems; k++){
                int best=prev[c][k];
                if(k>0 && W[i-1]<=c && prev[c-W[i-1]][k-1]>NEG)
                    best=max(best, prev[c-W[i-1]][k-1]+V[i-1]);
                cur[c][k]=best;
            }
        }
        swap(prev, cur);
    }
    int ans=NEG; for(int c=0; c<=Kcap; c++) ans=max(ans, prev[c][Kitems]);
    return ans;
}
```

4.3 2D Knapsack (Weight and Volume)

Description. Each item has weight w_i , volume u_i , value v_i . Maximize value under W and U .

State. $dp[i][w][u]$ = best value using first i items with caps w, u .

Recurrence.

$$dp[i][w][u] = \max \left(dp[i-1][w][u], dp[i-1][w-w_i][u-u_i] + v_i \right)$$

(valid when $w_i \leq w, u_i \leq u$). **Base.** $dp[0][*][*] = 0$.

(2 layers, watch order).


```

int knap_2d(const vector<int>& Wt, const vector<int>& Vol,
            const vector<int>& Val,
            int W, int U){
    const int n=Wt.size(), NEG=-1e9;
    vector prev(W+1, vector<int>(U+1, 0)), cur=prev;
    for(int i=1; i<=n; i++){
        cur=prev;
        for(int w=0; w<=W; w++){
            for(int u=0; u<=U; u++){
                int best=prev[w][u];
                if(Wt[i-1]<=w && Vol[i-1]<=u)
                    best=max(best, prev[w-Wt[i-1]][u-Vol[i-1]] + Val[i-1]);
                cur[w][u]=best;
            }
        }
        swap(prev, cur);
    }
    int ans=0;
    for(int w=0; w<=W; w++)
        for(int u=0; u<=U; u++)
            ans=max(ans, prev[w][u]);
    return ans;
}

```

4.4 Count Grid Paths with at Most K Turns

Description. From $(0, 0)$ to $(R-1, C-1)$ moving only Right/Down; count paths using at most K turns (direction changes).

State. Let $R[i][j][t]$ be #paths to (i, j) ending with a Right move using t turns, and $D[i][j][t]$ similarly for Down.

Recurrence.

$$\begin{aligned}
 R[i][j][t] &= R[i][j-1][t] + \mathbf{1}_{t>0} \cdot D[i][j-1][t-1], \\
 D[i][j][t] &= D[i-1][j][t] + \mathbf{1}_{t>0} \cdot R[i-1][j][t-1].
 \end{aligned}$$

Base. Start from $(0, 0)$: $R[0][1][0] = 1$ (first step right), $D[1][0][0] = 1$ (first step down) if in bounds.

Answer. $\sum_{t=0}^K (R[R-1][C-1][t] + D[R-1][C-1][t])$.
(bounds-checked).

```

long long paths_with_turns(int Rn, int Cn, int K){
    vector R(Rn, vector(Cn, vector<long long>(K+1, 0))),
        D=R;
    if(Cn>1) R[0][1][0]=1;
    if(Rn>1) D[1][0][0]=1;
    for(int i=0; i<Rn; i++) for(int j=0; j<Cn; j++){
        if(i==0 && j<=1) continue;
        if(j==0 && i<=1) continue;
        for(int t=0; t<=K; t++){
            if(j>0){
                R[i][j][t] += R[i][j-1][t];
                if(t>0) R[i][j][t] += D[i][j-1][t-1];
            }
            if(i>0){
                D[i][j][t] += D[i-1][j][t];
                if(t>0) D[i][j][t] += R[i-1][j][t-1];
            }
        }
    }
    long long ans=0;
    for(int t=0; t<=K; t++) ans += R[Rn-1][Cn-1][t] + D[Rn-1][Cn-1][t];
    if(Rn==1 && Cn==1) ans=1; // zero-move path
    return ans;
}

```

4.5 Traveling Salesman (Bitmask DP)

Problem. Given a complete weighted graph on n vertices, find the minimum-cost tour that begins at a fixed vertex s , visits every vertex exactly once, and returns to s .

State. Represent a visited set S by an n -bit integer $mask$, where bit v is set exactly when $v \in S$. Define

$$dp[mask][v] = \text{minimum cost of a path that starts at } s, \\ \text{visits exactly the vertices in } mask, \text{ and ends at } v.$$

Only states with $s, v \in mask$ are valid. The mask is sufficient because future choices depend only on the current endpoint and which vertices remain unvisited, not on the order used to reach the state.

Base Case.

$$dp[1 \ll s][s] = 0,$$

and all other states initially have value $+\infty$.

Recurrence. The vertex immediately before v must be some $u \in mask \setminus \{v\}$. Therefore:

$$dp[mask][v] = \min_{u \in mask \setminus \{v\}} (dp[mask \setminus \{v\}][u] + w[u][v]).$$

Equivalently, a forward transition appends an unvisited vertex v :

$$dp[mask \cup \{v\}][v] = \min(dp[mask \cup \{v\}][v], dp[mask][u] + w[u][v]).$$

Answer. After every vertex has been visited, close the tour by returning to s :

$$\min_{v \neq s} (dp[(1 \ll n) - 1][v] + w[v][s]).$$

Bottom-Up Bitmask DP.

```
long long tsp(const vector<vector<int>>& w, int s = 0) {
    int n = w.size();
    int full = 1 << n;
    const long long INF = (1LL << 60);

    vector<vector<long long>> dp(
        full, vector<long long>(n, INF)
    );
    dp[1 << s][s] = 0;

    for (int mask = 0; mask < full; ++mask) {
        if (!(mask & (1 << s))) continue;

        for (int u = 0; u < n; ++u) {
            if (dp[mask][u] == INF) continue;

            for (int v = 0; v < n; ++v) {
                if (mask & (1 << v)) continue;
                int nextMask = mask | (1 << v);
                dp[nextMask][v] = min(
                    dp[nextMask][v],
                    dp[mask][u] + w[u][v]
                );
            }
        }
    }

    long long answer = INF;
    for (int v = 0; v < n; ++v)
        if (v != s)
            answer = min(answer, dp[full - 1][v] + w[v][s]);

    return n == 1 ? 0 : answer;
}
```

Correctness. Proceed by induction on the number of set bits in $mask$. The base state is the zero-cost path containing only s . For a larger state ending at v , every valid path has a unique penultimate vertex u ; removing v produces exactly the smaller state in the recurrence. Minimizing over all possible u therefore produces the optimal path for the state. The final minimization adds the only missing edge, from the last vertex back to s , so it yields the optimal Hamiltonian tour.

Complexity. There are $n2^n$ states and at most n outgoing transitions per state:

$$O(n^2 2^n) \text{ time, } \quad O(n 2^n) \text{ space.}$$

This is practical only for small n , commonly around $n \leq 20$.

4.6 Minimum Cost to Merge Stones (LeetCode 1000)

Problem. Given n consecutive stone piles and an integer $K \geq 2$, one operation merges exactly K consecutive piles into one pile and costs the total number of stones in those piles. Find the minimum cost of merging all piles into one, or return -1 if this is impossible.

Feasibility Condition. Every operation reduces the number of piles by exactly $K - 1$. If t operations turn n piles into one, then

$$n - t(K - 1) = 1.$$

Consequently, a solution exists exactly when

$$(n - 1) \bmod (K - 1) = 0.$$

Prefix Sums. Let

$$\text{prefix}[i + 1] = \text{prefix}[i] + \text{stones}[i].$$

The sum of the interval $[i, j]$ is then available in constant time:

$$\text{sum}(i, j) = \text{prefix}[j + 1] - \text{prefix}[i].$$

State. An interval of length L can be reduced as far as possible to

$$1 + ((L - 1) \bmod (K - 1))$$

piles. Define $dp[i][j]$ as the minimum cost of reducing $\text{stones}[i..j]$ to this smallest attainable number of piles.

Base Case. One pile needs no operations:

$$dp[i][i] = 0.$$

Recurrence. Split the interval after m . It is sufficient to try $m = i, i + (K - 1), i + 2(K - 1), \dots$, because each left interval then has length congruent to 1 (mod $K - 1$) and can be reduced to one pile:

$$dp[i][j] = \min_{\substack{i \leq m < j \\ m - i \equiv 0 \pmod{K - 1}}} (dp[i][m] + dp[m + 1][j]).$$

If the whole interval can now become one pile, the final operation merges its K remaining piles and costs the interval sum:

$$dp[i][j] += \text{sum}(i, j) \quad \text{when} \quad (j - i) \bmod (K - 1) = 0.$$

Bottom-Up Interval DP.

```
long long mergeStones(const vector<int>& stones, int K) {
    int n = stones.size();
    if (n == 0) return 0;
    if ((n - 1) % (K - 1) != 0) return -1;

    vector<long long> prefix(n + 1, 0);
    for (int i = 0; i < n; ++i)
        prefix[i + 1] = prefix[i] + stones[i];

    const long long INF = (1LL << 60);
    vector<vector<long long>> dp(n, vector<long long>(n, 0));

    for (int len = 2; len <= n; ++len) {
        for (int i = 0; i + len <= n; ++i) {
            int j = i + len - 1;
            dp[i][j] = INF;

            for (int m = i; m < j; m += K - 1)
                dp[i][j] = min(
                    dp[i][j],
                    dp[i][m] + dp[m + 1][j]
                );
        }
    }

    return dp[0][n - 1];
}
```

```

    );
    if ((len - 1) % (K - 1) == 0)
        dp[i][j] += prefix[j + 1] - prefix[i];
    }
}

return dp[0][n - 1];
}

```

Correctness. Consider intervals in increasing order of length. For a fixed interval, every valid merge plan must first reduce suitable subintervals independently; the split recurrence tries every possible boundary at which the left side can become one pile. By the induction hypothesis, both subinterval costs are optimal. If the interval is eligible to become one pile, every plan must perform one final merge whose cost is exactly its total sum, which the recurrence adds. Thus $dp[i][j]$ is optimal for every interval, including the full array.

Complexity. There are $O(n^2)$ intervals and $O(n/(K - 1))$ relevant split points per interval:

$$O\left(\frac{n^3}{K-1}\right) = O(n^3) \text{ time,} \quad O(n^2) \text{ space.}$$

Nitpicks. (i) For maximization with impossible states, use a clear $-\infty$ sentinel (e.g., -10^9) and guard additions. (ii) Rolling arrays reduce memory for item-indexed DPs. (iii) For path-counting, prefer **long long**/mod. (iv) In turn-DP, initialize the first step explicitly; most mistakes there are off-by-one on the start cells.

5 Pseudo-Polynomial Dynamic Programming

5.1 Overview

Typical examples include 0/1 knapsack and coin change: their running times are polynomial in a numeric target such as the capacity or amount, rather than in the number of bits needed to encode that target.

5.2 Counting (r, ℓ) -Spaced Subsets in $O(rn \log n)$ Time

We are given a set S of n distinct non-negative integers, and two integers $r, \ell \geq 0$. A subset $S' \subseteq S$ is called (r, ℓ) -spaced if:

1. the sum of all elements in S' is exactly r , and
2. for any two distinct $a, b \in S'$, we have $|a - b| \geq \ell$.

We aim to count the number of such subsets in time $O(rn \log n)$.

Preprocessing

Sort the elements of S into an array

$$A[0], A[1], \dots, A[n-1],$$

in non-decreasing order. Sorting costs $O(n \log n)$.

Dynamic Programming Definition

For indices $i \in \{0, \dots, n\}$ and sums $j \in \{0, \dots, r\}$, define:

$$x(i, j) = \text{number of } (j, \ell)\text{-spaced subsets using only } A[i], A[i+1], \dots, A[n-1].$$

The desired answer is $x(0, r)$.

Base Cases.

$$x(n, 0) = 1, \quad x(n, j) = 0 \text{ for all } j > 0.$$

Transition: Optimal Substructure

For each state (i, j) , we consider two possibilities:

Case 1: Use element $A[i]$. This is only possible if $A[i] \leq j$. If we include $A[i]$, no further chosen elements may lie in the interval $[A[i], A[i] + \ell)$. Define $f(i)$ as the smallest index $k > i$ such that:

$$A[k] - A[i] \geq \ell.$$

This index can be found in $O(\log n)$ time using binary search over the sorted array.

If we choose $A[i]$, the new target sum becomes $j - A[i]$ and the next allowed index is $f(i)$, giving:

$$x(i, j) \supset x(f(i), j - A[i]).$$

Case 2: Do not use element $A[i]$. We simply skip $A[i]$, giving:

$$x(i, j) \supset x(i + 1, j).$$

DP Recurrence

Thus the recurrence is:

$$x(i, j) = \begin{cases} x(i + 1, j) + x(f(i), j - A[i]) & \text{if } A[i] \leq j, \\ x(i + 1, j) & \text{if } A[i] > j. \end{cases}$$

Time Complexity

There are $(n + 1)(r + 1) = O(rn)$ DP states. Each state requires:

- $O(1)$ time to combine solutions,
- $O(\log n)$ time to compute $f(i)$ using binary search.

Therefore, the total DP time is:

$$O(rn \log n).$$

Including sorting, the overall running time remains:

$$\boxed{O(rn \log n)}.$$

Thus, we obtain an algorithm that counts all (r, ℓ) -spaced subsets in time $O(rn \log n)$, as required.